Kinematic Modelling and MATLAB-Based Analysis of the Dobot Magician Lite Robot Arm

1 Introduction

Robot manipulators are widely used across industrial, medical and educational domains because of their precision, repeatability and programmability [6]. Common applications include pick-and-place operations, assembly lines, welding, laboratory automation, and prototyping. In educational robotics platforms, systems such as the Dobot Magician Lite are used for teaching robotics concepts because they combine low cost with multi-axis motion capability.

The Dobot Magician Lite is a desktop robotic manipulator with four revolute joints (J1–J4), forming an articulated robotic arm structure [8]. It operates in both point and Cartesian coordinate systems, enabling flexible motion control and trajectory planning. The robot supports multiple motion modes [9] such as point-to-point (PTP), linear (MOVL), and arc-based trajectories, making it suitable for both academic and practical applications [8].

From a classroom perspective, the Dobot Magician Lite can be categorized as a serial articulated robot arm. Compared to parallel manipulators, serial manipulators have a simpler structure and larger workspace, but their accuracy is lower without and accumulated positioning errors. Compared to SCARA robots, the Dobot provides greater vertical flexibility but may have reduced speed and precision in planar tasks.

The Dobot Magician Lite is particularly suitable for educational use because of its low cost, ease of setup, and integrated software and hardware [8]. However, limitations include restricted payload capacity (250 g), limited reach (340 mm), and lower repeatability compared to industrial robots. These characteristics make the Dobot of this project was to develop an analytical and MATLAB-based model of the Dobot Magician Lite and to evaluate its forward kinematics, inverse kinematics, Jacobian, singularities and validation behaviour.

List of Symbols

- θ_1 – base rotation angle
- θ_2 – shoulder joint angle
- θ_3 – elbow joint angle
- θ_4 – end-effector rotation angle
- L_1, L_2, L_3 – main arm link lengths

2 Kinematic Modelling

2.1 Robot configuration and assumptions

The kinematic model therefore treats the robot as a 4R serial manipulator, defined by its Denavit-Hartenberg parameters, forward and inverse kinematics, and a controlling tool orientation.

The Dobot Magician Lite kinematic model includes a parallelogram mechanism [14] that keeps the end-effector orientation approximately constant relative to the base. This mechanism ensures that behaviour through an equivalent passive transformation ($-\theta_2 - \theta_3$). In present kinematic model is treated as an actuated revolute joint with four revolutes, but only as a modelling device used to preserve the geometric constraint.

Figure 1: Appearance of the Dobot Magician Lite showing the physical arrangement of the four revolute joints used in the kinematic model. Adapted from [8].

Figure 2: Denavit-Hartenberg frame assignment used for the kinematic model of the Dobot Magician Lite manipulator.

Table 1: Denavit-Hartenberg parameter table derived from the frame assignment in Figure 2.

Link	α_i (deg)	a_i (mm)	d_i (mm)	θ_i
1 - Base	90°	0	138	θ_1
2 - Rear Arm	150	135	0	θ_2
3 - Fore Arm	150	147	0	θ_3
4 - uncontrollable link	0	90°	0	$-\theta_2 - \theta_3$
5 - End-Effector	0	60	0	θ_4

The coordinate frames were assigned according to the standard Denavit-Hartenberg convention [8]. Each z-axis was aligned with the axis of rotation of the corresponding revolute joint, and the x-axis was chosen along the common normal between adjacent joint axes. The physical robot has four revolute joints; however, an additional passive transformation was introduced only to capture the parallelogram constraint that keeps the end-effector orientation approximately consistent. This passive term is not an independent joint and does not increase the number of actuated degrees of freedom.

The Denavit-Hartenberg convention provides a systematic framework for assigning link parameters and deriving homogeneous transformations for serial manipulators [2, 3].

2.3 Forward kinematics

The forward kinematics of the Dobot Magician Lite were derived using the Denavit-Hartenberg convention. A homogeneous transformation matrix was obtained for each link, and the overall end-effector pose was computed by multiplying these matrices in the prescribed frame. The overall transformation from the base frame to the end-effector frame was then extracted from the resulting product. For the equivalent kinematic model used in this project, the total transformation takes the form:

$$T_{04} = A_1 A_2 A_3 A_4$$

where A_i denotes the passive transformation contributed to represent the parallelogram constraint in the physical robot. This passive term is included only to preserve the geometric behaviour of the mechanism and does not correspond to an additional actuated degree of freedom.

The position of the end-effector in the base frame provides both the orientation of the end-effector and the Cartesian position. The orientation is given by the rotation terms of this transformation were used here to derive the inverse kinematics expressions and to construct the Jacobian for velocity and singularity analysis. TODO: This is wrong as we need L1 which is non-zero

$$T_{04} = \begin{bmatrix} c\theta_1(c\theta_2 + c\theta_3) & -s\theta_1 & c\theta_1(c\theta_2 L_3 + c\theta_3 L_3 + L_2) \\ s\theta_1(c\theta_2 + c\theta_3) & c\theta_1 & s\theta_1(c\theta_2 L_3 + c\theta_3 L_3 + L_2) \\ 0 & 1 & 0 \\ 0 & 0 & h + L_2 s\theta_2 + L_3 s\theta_2 + \theta_3 \\ 0 & 0 & 1 \end{bmatrix}$$

2.4 Jacobian and singularity analysis

The Jacobian matrix relates joint rates to end-effector motion. In this work, the geometric Jacobian was used to study the relationship between joint velocities and the linear and angular velocity of the end-effector, while an analytical form was considered for singularity analysis. The Jacobian was used for velocity analysis [7]. Jacobian-based analysis is particularly useful for identifying configurations in which the manipulator loses mobility or becomes difficult to control.

Geometric Jacobian

The geometric Jacobian relates the joint velocities to the linear and angular velocity of the end-effector

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = J_g(q)\dot{q}$$

where J_g denotes the geometric Jacobian formulated or computed for the parallelogram constraint in the physical robot. This passive term is included only to preserve the geometric behaviour of the mechanism and does not correspond to an additional actuated degree of freedom.

The Jacobian matrix for the four-joint kinematic model provides both the orientation of and effective and the Cartesian position. The orientation is given by the rotation terms of this transformation were used here to derive the inverse kinematics expressions and to construct the Jacobian for velocity and singularity analysis. TODO: This is wrong as we need L1 which is non-zero

$$J_v(q) = \begin{bmatrix} -c\theta_1(c\theta_2 + c\theta_3) & -c\theta_1(c\theta_2 L_3 + c\theta_3 L_3 + L_2) & -c\theta_1(c\theta_3 L_3 + L_2) & 0 \\ -s\theta_1(c\theta_2 + c\theta_3) & -s\theta_1(c\theta_2 L_3 + c\theta_3 L_3 + L_2) & -s\theta_1(c\theta_3 L_3 + L_2) & 0 \\ 0 & L_2 c\theta_2 + L_3 c\theta_2 + \theta_3 & c\theta_3 L_3 + L_2 & 0 \end{bmatrix}$$

The base offset h appears in the forward kinematics of z, but it does not appear in the Jacobian because it is a constant term. The corresponding angular velocity mapping is

$$J_\omega(q) = \begin{bmatrix} -s\theta_1(c\theta_2 + c\theta_3) & -s\theta_1(c\theta_2 + c\theta_3) & -s\theta_1(c\theta_2 + c\theta_3) & 0 \\ c\theta_1(c\theta_2 + c\theta_3) & c\theta_1(c\theta_2 + c\theta_3) & c\theta_1(c\theta_2 + c\theta_3) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Singularities

A singularity occurs when the determinant of the analytical Jacobian becomes zero:

$$\det(J) = L_2 L_3 \sin(\theta_3) = 0$$

This expression shows that singularities occur when either $\sin(\theta_3) = 0$ at $\pi = 0$. The condition $\sin(\theta_3) = 0$ corresponds to the arm being fully extended or fully folded, so that the links become collinear. The condition $\pi = 0$ corresponds to the wrist lying on the base axis, where the manipulator loses motion authority in certain horizontal directions. Near these configurations, the velocity mapping becomes ill-conditioned and large joint velocities may be required to generate small end-effector motions [8, 7].

2.5 Inverse kinematics

A geometric approach was used for solving the inverse kinematics problem. This method was chosen due to the relatively simple structure of the robot and the availability of clear geometric relationships between joint angles and end-effector position.

The inverse kinematics solution was derived by:

- First solving for the base rotation using $\theta_1 = \mathrm{atan2}(y, x)$
- Reducing the 3D problem into a 2D planar problem in the (r, z) plane
- Applying trigonometric relationships and cosine rules to determine θ_2 and θ_3
- Considering both elbow-up and elbow-down configurations

This approach was preferred over numerical methods due to its computational efficiency, analytical clarity, and suitability for real-time applications.

2.3 Dynamic modelling equation

The dynamic behaviour of the manipulator was considered using the standard rigid-body robot dynamics formulation [7]

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$$

where τ is the vector of joint torques, $M(q)$ is the inertia matrix, $C(q, \dot{q})$ represents Coriolis and centrifugal effects, and $G(q)$ is the gravity vector.

In this present project, the dynamic model was used primarily as a conceptual and modelling level to extend the kinematic model developed in MATLAB. Simulation and control implementation were not carried out in this work, but the model provides a useful foundation for future work on torque-based control, inertia tensors from the manufacturer documentation [8]. Therefore, the work focused on establishing the kinematic and geometric foundation for modelling how such terms would be integrated into the MATLAB model for future extension.

4 MATLAB Implementation

The robot model was implemented in MATLAB using a modular approach. Individual functions were created for the Denavit-Hartenberg transformations, forward kinematics, inverse kinematics, Jacobian computation, singularity analysis, and workspace validation. Symbolic computation was used where appropriate to derive compact expressions for the forward kinematics and Jacobian, which were then evaluated numerically during testing [1].

Joint limits were implemented as described in the manufacturer position could be modelled as equal, the corresponding joint angle was set to zero. Workspace boundary were kinematic, and the resulting configuration could then be verified through forward kinematics. Additional validation checks were included to detect unreachable targets and singular configurations.

5 Testing Framework

A structured testing framework was used to validate the correctness of the mathematical model and its MATLAB implementation. Forward kinematics were validated by comparing the computed end-effector position against the resulting end-effector position with the expected geometric outcome. Inverse kinematics were validated by joint reconstruction from known configurations and substituting the solution back into the forward kinematics equations.

Additional tests were used to verify singularity detection and workspace rejection for unreachable targets. Together, these checks ensured internal consistency between the analytical derivations and the computational implementation.

6 Results and Validation

The MATLAB model was validated using forward-kinematics, inverse-kinematics and singularity test cases. Forward kinematics was checked by comparing computed end-effector positions with expected geometric outcomes, while inverse kinematics was verified by reconstructing joint configurations and comparing Cartesian positions. Additional tests were used to confirm singularity detection and rejection of unreachable targets.

Table 2: Validation of forward and inverse kinematics test results table

Test case	Input	Result
Forward kinematics	$q = (0.5236, 0.7854, 0, 0)$ rad	$(x, y, z) = (0.1837, 0.3196, 0.5000)$ m
Inverse kinematics	Target $(0.2000, 0.0000, 0.4000)$ m	$q = (0.5236, 0.7854, 0, 0)$ rad, $\det(J) = 0$
Singularity test	$\theta_3 = 0$	$\det(J) = 0$
Unreachable target	Outside valid/ invalid cases	Rejected

The validation results show good agreement between the analytical model and the MATLAB implementation. In particular, the consistency between the forward kinematics output and the inverse kinematics reconstruction confirms that the position mapping is internally correct. The singularity and unreachable target tests further demonstrate that the implementation correctly handles important edge cases rather than only the nominal case.

Figure 3: Successful inverse kinematics solution with the target in reachable workspace.

Figure 4: Top-view geometry used to compute the base joint angle θ_1.

Figure 5: Planar geometric construction used for the inverse kinematics derivation.

Figure 6: Elbow-down configuration used for the inverse kinematics derivation.

Figure 7: Planar geometric construction used for the inverse kinematics derivation.

Figure 8: Elbow-up configuration used for the inverse kinematics derivation.

Figure 9: Planar geometric construction used for the inverse kinematics derivation.

7 Discussion and Conclusion

This project developed and validated a kinematic model of the Dobot Magician Lite using the Denavit-Hartenberg framework and implemented forward and inverse kinematics in MATLAB. Forward and inverse kinematics were derived analytically, the Jacobian was used for velocity and singularity analysis, and simulation results were used to validate the analytical model. The resulting MATLAB implementation provides a structured foundation for further work on robot motion, workspace validation and trajectory planning.

One of the main technical difficulties was representing the physical parallelogram mechanism of the robot within a standard serial kinematic model. This was addressed by replacing the closed-chain behaviour with an equivalent open-chain model and treating the passive transformation as a constraint that preserves orientation centered without introducing an additional actuated degree of freedom.

Despite these limitations, the final model provides a robust analytical and computational framework for studying the motion of the Dobot Magician Lite. The results demonstrate that the model is suitable for forward kinematics, inverse kinematics, singularity detection and validation testing in MATLAB. Future extensions could include trajectory generation, workspace boundary mapping, dynamic simulation, torque-based control, inertia tensors from the manufacturer documentation, and closed-loop control in order to improve realism and practical applicability.

Figure 8: Principal dimensions of the Dobot Magician Lite used to support the geometric assumptions of the model. Adapted from [8].

8 References

References

[1] P. Corke. *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*. Springer Tracts in Advanced Robotics. 2nd edition, 2017.

[2] J. J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson Education, Upper Saddle River, 3rd edition, 2005.

[3] J. Denavit and R. S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22:215–221, 1955.

[4] Dobot. *Dobot Magician Lite User Guide (DobotStudio)*. n.d. [Online]. Available: https://download.dobot.cc/product-manual/magician-lite/DobotStudio%20Magician%20Lite%20User%20Guide(en).pdf.

[5] Dobot. Firmware and apply inverse kinematics to robot arm. 2023. [Online]. Available: https://ask.dobotservice.com/help/public/article/firmware-and-apply-inverse-kinematics-to-robot-arm.html.

[6] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, Planning and Control*. Springer, London, 2009.

[7] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, Hoboken, 2005.